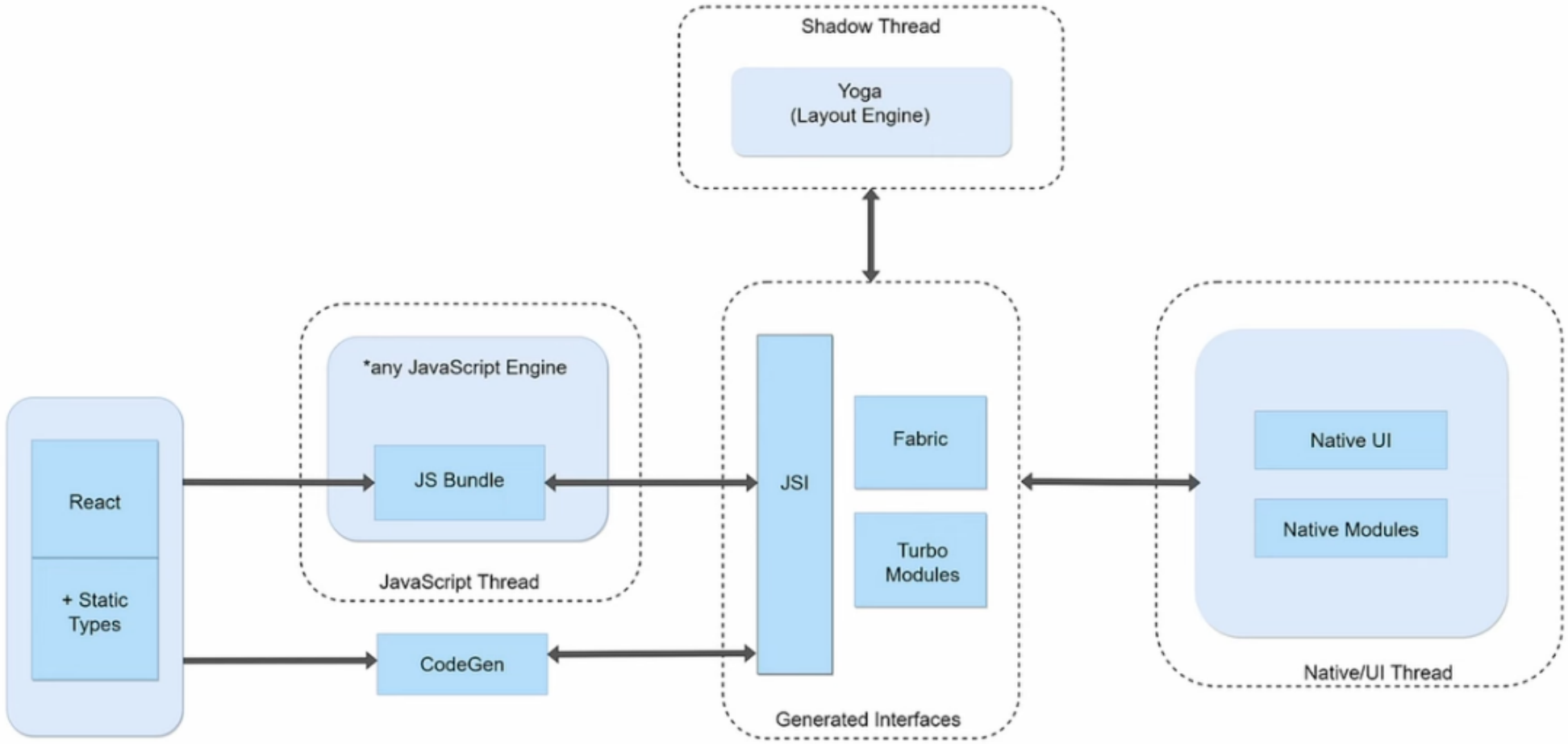
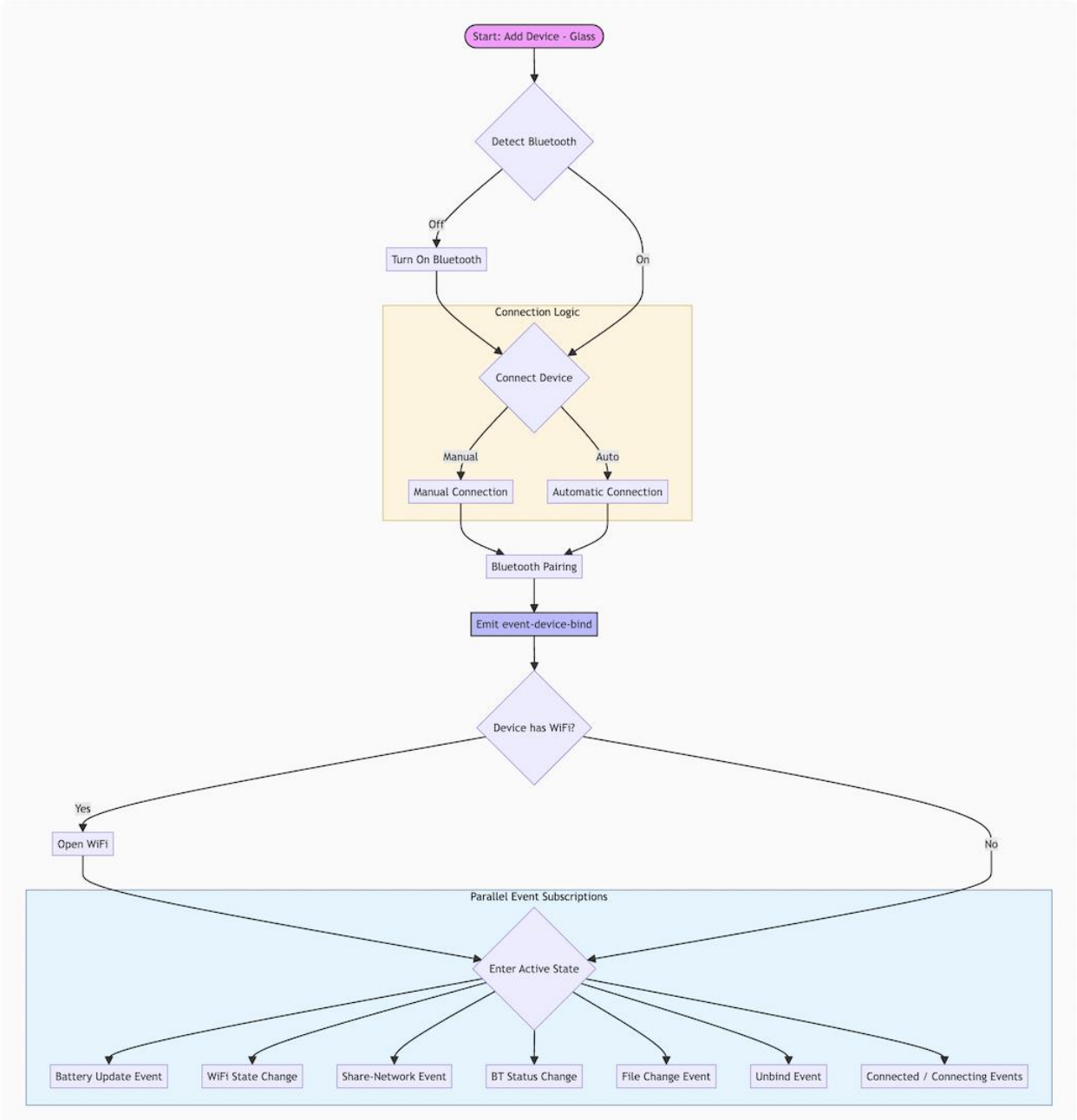
React VS React-Native


Shadow Thread
Yoga
(Layout Engine)
*any JavaScript Engine
JS Bundle
JavaScript Thread
React
+ Static
Types
CodeGen
JSI
Fabric
Turbo
Modules
Generated Interfaces
Native UI
Native Modules
Native/UI Thread

📱 Device Connection Flow - Glass Bluetooth Setup



1. New vs Old Architecutre Comparion(Fabric is React Native's new rendering system, React Native 0.68, **Version 0.76 default**)
2. The Rendering: DOM VS Native Views
3. Threading Architecture
4. Styling: Yoga vs Css
5. View Flattening
6. Navigation
7. Lifecycle
8. Engine
9. Timer(Web VS React-Native VS Node)
10. Build and Deployment

🏗️ React Native Rendering Architecture

Old Architecture (Pre-Fabric / "Paper")

```
┌─────────────────────────────────────────┐
│              JavaScript Thread          │
│  ┌──────────┐      ┌──────────┐         │
│  │ React Code │ ──→ │ React Native │     │
│  │ Components │     │   Bridge   │      │
│  └──────────┘      └──────────┘         │
└──────────────────────┬──────────────────┘
                       │
                       ▼
                JSON Messages (Async)
                       │
                       ▼
┌─────────────────────────────────────────┐
│              Native Thread              │
│  ┌──────────┐      ┌──────────┐         │
│  │ Shadow Tree │ ──→ │ Native Views │   │
│  │ (Layout)   │     │ (UIView/View) │  │
│  └──────────┘      └──────────┘         │
└─────────────────────────────────────────┘
```

Problems with Old Architecture:

1. **Asynchronous Bridge** - All communication serialized to JSON
2. **Three Threads** - JS, Shadow (layout), Native UI
3. **No Synchronous Access** - Can't directly measure views
4. **Performance Bottlenecks** - Bridge is a bottleneck for complex UIs

---

React Native: New Architecture (Fabric + TurboModules)

```
┌─────────────────────────────────────────────────────────┐
│ JavaScript Thread (JSI - JavaScript Interface)          │
│                                                         │
│  ┌──────────┐                                           │
│  │ React Core │                                         │
│  └──────────┘                                           │
│       │                                                 │
│       │ Direct C++ calls (No Bridge! ✨)                │
│       │ Synchronous access to native                    │
│       ▼                                                 │
│  ┌──────────────────────────────────────────┐           │
│  │ JSI (C++ Host Objects)                   │           │
│  │ - No serialization                       │           │
│  │ - Direct memory access                   │           │
│  │ - 10-100x faster ✅                      │           │
│  └──────────────────────────────────────────┘           │
└──────────────────────────┬──────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────┐
│ Fabric (New Renderer)                                    │
│ - Concurrent rendering                                   │
│ - Priority-based updates                                 │
│ - Better interruptibility                                │
└──────────────────────────────────────────────────────────┘
```

Synchronous Layout and Effects

Web Rendering vs. React-Native Fabric

🌐 React Web Rendering

React Web has a **completely different architecture** because it renders to the brow

```
┌──────────────────────────────────────────────────────────────┐
│                        JavaScript Thread                     │
│                                                              │
│  ┌──────────────────┐                                        │
│  │ React Code       │                                        │
│  │ Components       │                                        │
│  └──────────────────┘                                        │
│          │                                                   │
│          │ React Reconciler                                  │
│          ▼                                                   │
│  ┌──────────────────┐      ┌──────────────────┐             │
│  │ Virtual DOM      │ ───► │ Browser DOM      │             │
│  │ (React Tree)     │ Diff │ (Real DOM)       │             │
│  └──────────────────┘      └──────────────────┘             │
│                                    │                         │
│                                    ▼                         │
│                            ┌──────────────────┐             │
│                            │ Browser Render   │             │
│                            │ Engine           │             │
│                            └──────────────────┘             │
└──────────────────────────────────────────────────────────────┘
```

New Architecture (Fabric)

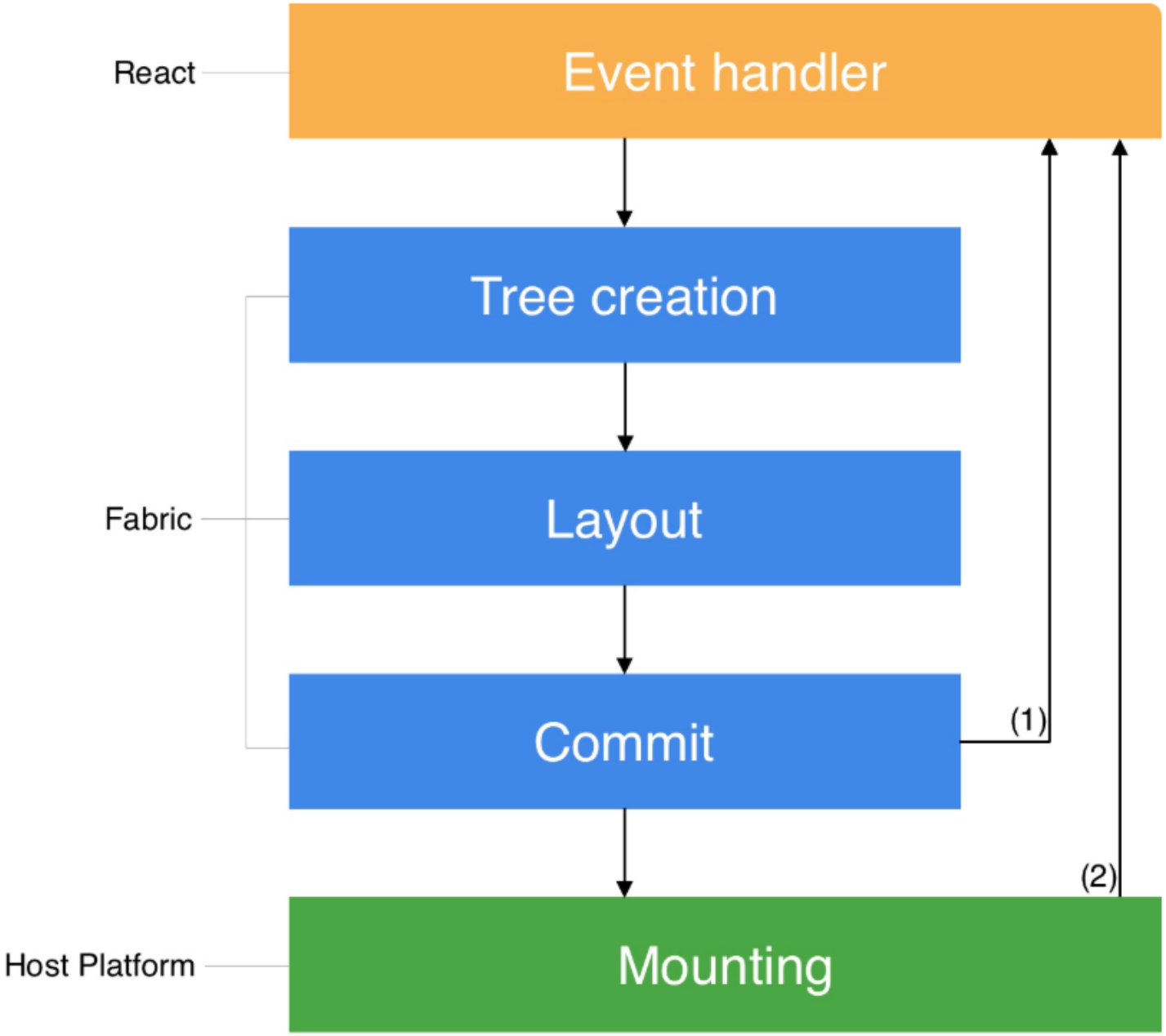
```
┌──────────────────────────────────────────────────────────────┐
│                        JavaScript Thread                     │
│                                                              │
│  ┌──────────────────┐                                        │
│  │ React Code       │                                        │
│  │ Components       │                                        │
│  └──────────────────┘                                        │
│          │                                                   │
│          │ JSI (JavaScript Interface)                        │
│          │ ↓ Synchronous, Direct C++ calls                   │
│          ▼                                                   │
│  ┌──────────────────┐      ┌──────────────────┐             │
│  │ Fabric           │ ───► │ Native Views     │             │
│  │ Renderer         │ C++  │ (UIView/View)    │             │
│  └──────────────────┘      └──────────────────┘             │
└──────────────────────────────────────────────────────────────┘
```

Key Improvements:

1. **JSI (JavaScript Interface)** - Direct C++ bindings, no bridge
2. **Synchronous** - Can synchronously call native methods
3. **Single Thread (Optional)** - Can render on any thread
4. **Better Performance** - No serialization overhead

The Rendering: DOM VS Native Views

Rendering Target	**Browser DOM** Virtual DOM diffs are reconciled into real DOM nodes — ** <img/>**. The browser's rendering engine (Blink, WebKit, Gecko) handles layout, paint, and compositing. **JSX→VDOM→DOM Nodes→Browser Paint**	**Platform Native Views** Renders to actual native UI components — **UIView** (iOS) / **android.view.View** (Android) . No WebView, no HTML. The OS composites the views directly via GPU. **JSX → React Element Tree→ React shadow Tree → Host View Tree(Native View)**



1. Events from core. For example: onLayout
2. Events from host platform. For example: onChange

Initial Render Pipeline

RENDER PHASE MAIN THREAD

Execute Component → Virtual DOM

function MyComponent() executes in V8/SpiderMonkey/JSC. Returns React Elements → builds the **Virtual DOM** (JS objects in memory). Also called the "Fiber Tree" internally.

RECONCILIATION MAIN THREAD

Diff Virtual DOM (no-op on first render)

On initial render: no previous tree to diff. React marks all Fiber nodes as needing placement. Builds the "effect list" of DOM operations to perform.

COMMIT PHASE MAIN THREAD

Create DOM Nodes & Append

React calls document.createElement(), sets attributes/properties, calls appendChild() to build the real DOM tree.

BROWSER PAINT MAIN THREAD + GPU

Layout → Paint → Composite

Browser CSS engine calculates layout (reflow), paints pixels to layers, and the GPU compositor assembles the final frame.

DOM element on screen

RENDER PHASE JS THREAD

Execute Component → Create React Shadow Node(React Host Components)

function MyComponent() executes on the JS thread via Hermes. Creates **React Element tree (JS objects),** then builds the **Shadow Tree (C++)** via JSI. The Shadow Tree lives in C++ memory.

COMMIT PHASE UI THREAD

Layout calculation(React Shadow Node) + Tree Promotion(New Tree -> Next Tree)

Yoga Layout Calculation

Yoga (C++ Flexbox engine) traverses the Shadow Tree, calculates layout: positions (x, y), sizes (width, height), via Flexbox algorithm. Produces absolute pixel coordinates for every node.

MOUNT PHASE UI THREAD

Transforms the React Shadow Tree(contains data from layout calculation) into a Host View Tree with rendered pixels on the screen

Tree Diffing(view Flattening) + Tree Promotion(Next Tree -> Render Tree) + View Mounting

Platform native views are instantiated: UIView / ViewGroup. Properties applied, views inserted into the native view hierarchy.

Native View on screen

React State Updates Pipeline

TRIGGER MAIN THREAD
setState() / Hooks Update
setState or useState setter called. React 18 automatically batches updates (even in async code).

RENDER PHASE MAIN THREAD
Re-execute Component → New Virtual DOM
Component re-renders, producing a new VDOM tree (Fiber nodes). This is the "work-in-progress" tree.

RECONCILIATION MAIN THREAD
VDOM Diff → Minimal DOM Operations
React's fiber reconciler diffs old vs new VDOM. Produces a list of DOM mutations: setAttribute, insertBefore, removeChild. Batches DOM operations for efficiency.

BROWSER REPAINT MAIN THREAD + GPU
Reflow → Repaint → Composite
Browser recalculates affected layout regions, repaints dirty areas, composites layers on GPU. useLayoutEffect runs synchronously before paint.

Updated DOM Element

TRIGGER JS THREAD
setState() / Hooks Update
setState or useState setter called. React schedules a re-render on the JS thread. Batching may combine multiple state updates.

RENDER PHASE JS THREAD
Re-execute Component → New React Shadow Tree
Component function re-executes. A new Shadow Tree is created in C++ memory. This tree represents the **desired** state of the UI.

COMMIT PHASE
Layout calculation
Tree Promotion(new tree -> next tree)

MOUNT UI THREAD
Tree Promotion(Next Tree -> Rendered Tree)
Tree Diffing
View Mounting

Update Native View

React Native Render State Updates (C++ state updates)

No C++ State Update Path on Web

The web has no concept of a "C++ shadow tree" that can be mutated independently of JavaScript. All React state changes must go through the JS main thread.
However, the browser does offer some mechanisms to offload work from JS:

CSS Transitions / Animations — GPU-composited, don't touch JS or layout.
transform + opacity — compositor-only properties, skip layout & paint entirely.
scroll — native browser scrolling is off-main-thread by default (compositor thread).
Web Animations API — declarative animations that run on the compositor.
OffscreenCanvas — WebGL/Canvas rendering in a Worker thread.

When C++ State is used only when the platform knows something React doesn't need to control . Eg ScrollView

User scrolls → iOS/Android reports offset (x: 0, y: 347)
↓
C++ State updated directly
↓
Used by measure() API internally
↓
React Element Tree is NOT affected

1. C++ state updates skip the "render phase" since React is not involved.
   Native → C++ (JS never touched)
2. The updates can originate and happen on any thread, including the main thread.

COMMIT PHASE

1. Read latest committed version of ShadowNode N
2. Clone N → N' with new state S
3. Attempt to commit N' to the tree
4. If commit succeeds → done
5. If commit fails (tree changed underneath) → retry from step 1

MOUNT PHASE

Tree Diff (old ↔ new) → Apply to Native Views

Same as normal state updates: diff old vs new Shadow Tree, produce mutations, apply to native views on UI thread.

Updated Native View(JS Thread never touched)

Basic Components Mapping

Web	React Native	Notes
`<div>`	`<View>`	Container
`<span>`, `<p>`	`<Text>`	**ALL text must be in `<Text>`** ⚠️
`<input>`	`<TextInput>`	Different props API
`<button>`	`<TouchableOpacity>` / `<Pressable>`	More touch options
`<img>`	`<Image>`	Requires `source` prop
`<a>`	`<TouchableOpacity>` + navigation	No `href`
`<ul>`, `<li>`	`<FlatList>` / `<ScrollView>`	Optimized for performance

View Flattening

```
Native View Hierarchy (Deep):

┌──────────────────────────┐
│ View (flex: 1)           │
│  ┌──────────────────────┐│
│  │ View (pad: 10)       │
│  │  ┌──────────────────┐│
│  │  │ View (m: 5)      │
│  │  │  ┌──────────────┐│
│  │  │  │   Text       │
│  │  │  └──────────────┘│
│  │  └──────────────────┘│
│  └──────────────────────┘│
└──────────────────────────┘

Depth: 4 levels
Native views: 3 Views + 1 Text = 4 views
```

With flattening (optimized):

```
Native View Hierarchy (Flat):

┌──────────────────────────┐
│ View                     │
│ (flex: 1 + pad: 10 + margin: 5)│
│  ┌──────────┐            │
│  │  Text    │            │
│  └──────────┘            │
└──────────────────────────┘

Depth: 2 levels
Native views: 1 View + 1 Text = 2 views
```

Layout-only nodes:

Layout Properties (Flattenable)

Category	Properties
Flexbox	flex, flexDirection, flexWrap, flexGrow, flexShrink, flexBasis, alignItems, alignSelf, alignContent, justifyContent
Dimensions	width, height, minWidth, minHeight, maxWidth, maxHeight
Spacing	margin*, padding* (all variants)
Positioning	position, top, right, bottom, left, start, end
Display	display (when 'flex' or 'none')

Visual Properties (NOT Flattenable)

Category	Properties
Background	backgroundColor
Border	border*, borderRadius, borderTopLeftRadius, etc.
Shadow (iOS)	shadowColor, shadowOffset, shadowOpacity, shadowRadius
Elevation (Android)	elevation
Opacity	opacity
Transform	transform, translateX, translateY, rotate, scale
Overflow	overflow: 'hidden' or 'scroll'
Accessibility	accessible, accessibilityLabel, etc.
Event Handlers	onPress, onLayout, etc.

Threading Architecture

Threading Architecture	**Single-Threaded (Main Thread)** JS execution, DOM manipulation, layout, and paint all share the **Main Thread**. Heavy computation blocks rendering. Web Workers provide off-thread JS but *cannot* access the DOM. **Offloading** Web Workers, requestIdleCallback, OffscreenCanvas	**Multi-Threaded (3+ Threads)** **JS Thread** runs Hermes/JSC — executes your React code. **UI Thread** (Main) — renders native views, handles touch. **Shadow Thread** — Yoga layout calculation (New Arch merges this into background work). **New Arch (Fabric)** JSI enables synchronous cross-thread calls, replacing the async Bridge.

Styling: Css Vs Yoga

Styling System	**CSS (Full Spec)** Full CSS cascade, selectors, media queries, pseudo-elements, Grid, Flexbox(flex-direction: row), animations, transitions. Styles are parsed by the browser's CSS engine and applied during layout/paint. Tooling: **CSS Modules** , **Tailwind, styled-components,** **CSS-in-JS** **Web default: content-box(**width(content-only) + padding + border**)**	**Yoga (Flexbox Subset in C++)** JS objects passed to the **Yoga** layout engine. Flexbox-only(default: column) — no cascade, no selectors, no CSS Grid, no pseudo-elements. All values are unitless (ios(pt), android(Density-independent Pixels (dp), web(pixels px)) all text **must** be wrapped in a <Text> StyleSheet.create() for optimization. Limitations: **No inheritance No cascade No media queries** (use Dimensions API) React: border-box, Width **= width + padding left + padding right + border left + border right**

Key Styling Differences

Feature	Web	React Native
Units	px, em, rem, %, vh, vw	**Only numbers** (density-independent pixels)
Flexbox	Optional	**Default layout** (flex: 1)
flex-direction	`row` (default)	`column` **(default)** ⚠️
Position	static, relative, absolute, fixed	**Only relative, absolute**
Box Model	content-box (default)	**border-box (default)**
Text styling	Can style any element	**Only** `<Text>` **can have text styles** ⚠️

Navigation

Navigation & Routing	**URL-Based Routing** Browser history stack via **History API**. Routes map to URL paths. Deep linking is native to the web. Back/forward managed by the browser. Libraries: **React Router** , **Next.js Router TanStack Router**	**Native Stack-Based Navigation(a stack of screens)** No URL bar. Navigation mimics native app patterns — **Stack, Tab, Drawer** navigators. Each screen is a native view controller with platform transitions and gestures. Libraries: **React Navigation Expo Router** (file-based, adds URL support)

Lifecycle

Component Lifecycle	**Standard React Lifecycle** **useEffect useLayoutEffect useMemo** — runs in a single environment. Page visibility via visibilitychange event. Cleanup on unmount is straightforward.	**React Lifecycle + App State** Same hooks, but with added **AppState** (active / background / inactive). Must handle: OS killing background processes, memory warnings, and native module lifecycle. **useFocusEffect** for screen visibility.

Engine

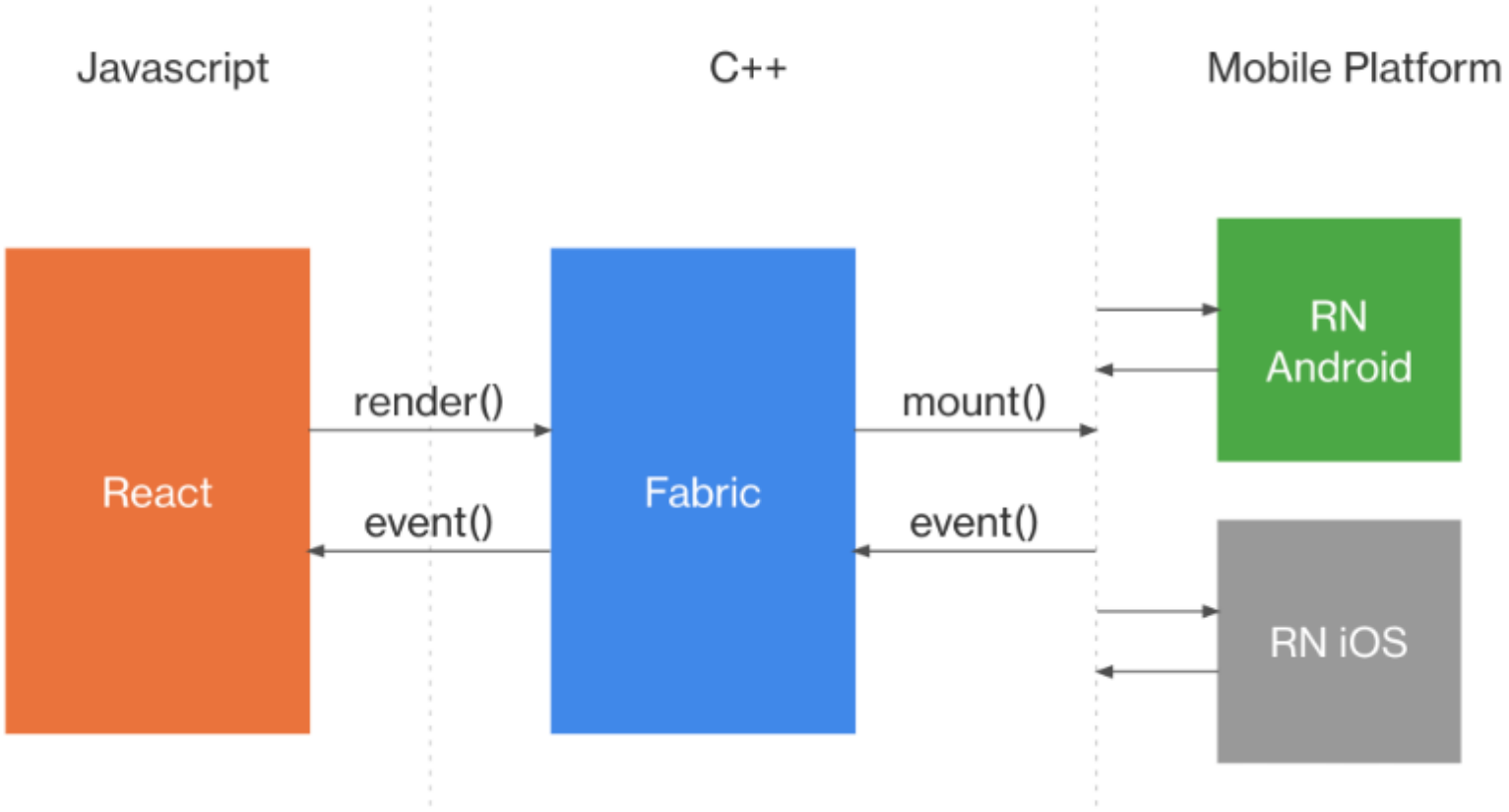
JavaScript Engine	**Browser JS Engine** **V8** (Chrome/Edge), **SpiderMonkey** (Firefox), **JavaScriptCore** (Safari). Full JIT compilation, mature garbage collection, Web APIs built in.	**Hermes (Default since RN 0.70)** **Hermes** — bytecode-compiled AOT(ahead of time) for fast startup and low memory. Optimized for mobile constraints. No full JIT on Android. Alternatively **JSC** on iOS (legacy). **Key advantage** Hermes precompiles JS → bytecode at build time, slashing cold start time.

1. Rendering Engine

Aspect	React Web	React Native
Target	DOM (Browser)	Native iOS/Android components
Rendering	HTML elements (`<div>`, `<span>`)	Native components (`<View>`, `<Text>`)
Styling	CSS files	JavaScript StyleSheet objects
Layout	Flexbox + CSS Grid + Block	**Flexbox only** (default)
Threading	Single thread	**JavaScript thread + UI thread** (bridge)

JS ↔ Native Communication

JS ↔ Native Communication	**Direct — No Bridge Needed** JS and DOM are in the same runtime. Calling document.createElement() is a synchronous engine API call. Web APIs (fetch, canvas, WebGL) are built into the browser.	**JSI / Bridge** **Old Arch** Async JSON Bridge — serializes all data between JS ↔ Native (bottleneck). **New Arch JSI** — C++ interface allowing synchronous, direct calls between JS and native. TurboModules + Fabric eliminate the bridge. **JS (Hermes)**↔ JSI ↔**C++ / Native**


Javascript
C++
Mobile Platform
React
render()
event()
Fabric
mount()
event()
RN
Android
RN iOS

Timer(Web VS React-Native VS Node)

```
## 1. Web Timer
// ┌──────────────────────────────┐
// │   Call Stack                 │
// │  (Synchronous Code)          │
// └──────────────┬───────────────┘
//                │
// ┌──────────────▼───────────────┐
// │  Microtask Queue             │
// │  - Promise.then()            │
// │  - queueMicrotask()          │
// │  - MutationObserver          │
// └──────────────┬───────────────┘
//                │
// ┌──────────────▼───────────────┐
// │  Render Pipeline             │
// │  - Style calculation         │
// │  - Layout                    │
// │  - Paint                     │
// └──────────────┬───────────────┘
//                │
// ┌──────────────▼───────────────┐
// │  Macrotask Queue             │
// │  - setTimeout                │
// │  - setInterval               │
// │  - I/O                       │
// └──────────────┬───────────────┘
//                │
// ┌──────────────▼───────────────┐
// │  requestAnimationFrame       │
// │  (before next paint)         │
// └──────────────┬───────────────┘
//                │
// ┌──────────────▼───────────────┐
// │  requestIdleCallback         │
// │  (when browser is idle)      │
// └──────────────────────────────┘
```

```
// ## 2. React Native Timers
// ### Event Loop Architecture
// ```
// JavaScript Thread          Bridge          Native Thread
// ──────────────────          ────────
// ┌──────────────────────────┐
// │  Synchronous Code        │
// └──────────────┬───────────┘
//                │
// ┌──────────────▼───────────┐
// │  Microtask Queue         │
// │  (Promise - uses         │
// │   setImmediate)          │
// └──────────────┬───────────┘
//                │
// ┌──────────────▼───────────┐
// │  setImmediate Queue      │
// └──────────────┬───────────┘
//                │
//                └──────────────────────> [Bridge Crossing]
//                                         │
//                                         ┌──────────────▼──────────┐
//                                         │  Native Processing      │
//                                         │  - Layout               │
//                                         │  - Render               │
//                                         └──────────────┬──────────┘
//                                                        │
//                                         ┌──────────────────────────┘
//                                         │
// ┌──────────────▼───────────┐
// │ requestAnimationFrame    │
// │ (after frame ready)      │
// └──────────────┬───────────┘
//                │
// ┌──────────────▼───────────┐
// │ setTimeout/setInterval   │
// └──────────────────────────┘
```

```
// ## 3. Node.js Timers

// ### Event Loop Architecture
// ```
//   ┌──────────────────────────────┐
// ┌─>│          timers              │ ← setTimeout, setInterval
// │  └──────────────┬───────────────┘
// │  ┌──────────────▼───────────────┐
// │  │     pending callbacks        │ ← I/O callbacks (deferred)
// │  └──────────────┬───────────────┘
// │  ┌──────────────▼───────────────┐
// │  │      idle, prepare           │ ← Internal use
// │  └──────────────┬───────────────┘
// │  ┌──────────────▼───────────────┐
// │  │           poll               │ ← I/O operations
// │  └──────────────┬───────────────┘
// │  ┌──────────────▼───────────────┐
// │  │           check              │ ← setImmediate callbacks
// │  └──────────────┬───────────────┘
// │  ┌──────────────▼───────────────┐
// │  │      close callbacks         │ ← socket.on('close', ...)
// │  └──────────────────────────────┘
// │
// │  ┌──────────────────────────────┐
// │  │  process.nextTick            │ ← Highest priority!
// │  │  (after each phase)          │
// │  └──────────────────────────────┘
// │
// │  ┌──────────────────────────────┐
// │  │  Promise Microtasks          │ ← After nextTick
// │  └──────────────────────────────┘
// │
// └──────────────────────────────────┘
```

Build & Deployment

Build & Deployment	**Bundle → CDN** **Vite, Webpack, esbuild** bundle JS/CSS/assets into static files. Deploy to any CDN or server. Instant updates — no app store review.	**Native Binary + JS Bundle** **Gradle** (Android) / **Xcode** (iOS) compile native code into an app binary containing the Hermes bytecode bundle. App Store / Play Store submission required. **OTA Updates** CodePush / Expo Updates can push JS-only changes without store review.

Problem 2: React's Unmount Timing

React's component lifecycle phases:

```
┌─────────────────────────────────────────────────┐
│  Component Lifecycle                            │
├─────────────────────────────────────────────────┤
│                                                 │
│  1. MOUNTING                                    │
│     └─ constructor() / useState initialization  │
│     └─ render()                                 │
│     └─ useEffect(() => { ... }, []) runs        │
│                                                 │
│  2. UPDATING (when props/state change)          │
│     └─ render()                                 │
│     └─ useEffect cleanup from previous render   │
│     └─ useEffect callback runs                  │
│                                                 │
│  3. UNMOUNTING                                  │
│     └─ useEffect cleanup functions run ✅       │
│     └─ Component removed from DOM ✅            │
│     └─ Internal fiber marked as "unmounted" ✅  │
│                                                 │
│  ⚠️ After unmounting:                           │
│     └─ setState calls throw warnings            │
│     └─ Refs become null                         │
│     └─ Event handlers still in memory if not    │
│        cleaned up (memory leak!)                │
└─────────────────────────────────────────────────┘
```